

Product Requirement Document

```
1  ## Task list
2  ```python
3  [
4      "main.py",
5      "canvas.py",
6      "tools.py",
7      "color_picker.py",
8      "file_manager.py"
9  ]
10 ```
11
12 ## Shared Knowledge
13 ```python
14 """
15 The 'Tkinter' library is used for creating the GUI of the application. It provides various widgets
16     ↳ like frames and canvases which are used in this project.
17 The 'Pillow' library is used for handling image files. It is used in the 'file_manager.py' for
18     ↳ saving and opening image files.
19 The 'main.py' file is the entry point of the application. It initializes all the other components
20     ↳ and starts the application loop.
21 """
22 ```
23
24 ## Anything UNCLEAR
25 The project requirements and implementation approach are clear. However, we need to ensure that
26     ↳ all team members are familiar with the Tkinter and Pillow libraries. If not, they may
27     ↳ need some time to learn these libraries before starting the development.
```

Engineer Given the provided file structure and function definitions, an Engineer agent requires only fundamental development skills to complete the development tasks. Due to the large number of files, we present only one auto-generated code file here.

Generated code (color_picker.py)

```
1 import tkinter as tk
2 from tkinter import colorchooser
3 class ColorPicker:
4     def __init__(self, root: tk.Tk, color: str = 'black'):
5         self.frame = tk.Frame(root)
6         self.color = color
7
8         self.color_button = tk.Button(self.frame, text='Color', command=self.select_color)
9         self.color_button.pack(fill='x')
10
11     def select_color(self):
12         color = colorchooser.askcolor()[1]
13         if color is not None:
14             self.color = color
15
16     def pack(self, **kwargs):
17         self.frame.pack(**kwargs)
18
19     def get_color(self):
20         return self.color
```

QA Engineer Upon receiving the code output from the Engineer, the QA Engineer generates unit test code and reviews it to identify and fix any bugs, ensuring high-quality software.

Generated Unit tests (test.py)

```
1 import unittest
2 import tkinter as tk
3 # You might want to move the ColorPicker class to another file, let's say color_picker.py
4 # from color_picker import ColorPicker
5
6 class TestColorPicker(unittest.TestCase):
7
8     def setUp(self):
9         self.root = tk.Tk()
10        self.cp = ColorPicker(self.root)
11
12    def tearDown(self):
13        # This is important to close the tk window after tests
14        self.root.destroy()
15
16    def test_initial_color(self):
17        # Test the default color during initialization
18        self.assertEqual(self.cp.get_color(), 'black')
19
20    def test_set_and_get_color(self):
21        # Test setting a new color and retrieving it
22        new_color = '#ffffff' # white color
23        self.cp.color = new_color
24        self.assertEqual(self.cp.get_color(), new_color)
25
26
27 if __name__ == '__main__':
28     unittest.main()
```

Output Ultimately, as shown in Figure 10, MetaGPT generates a functional application named “Drawing App”.

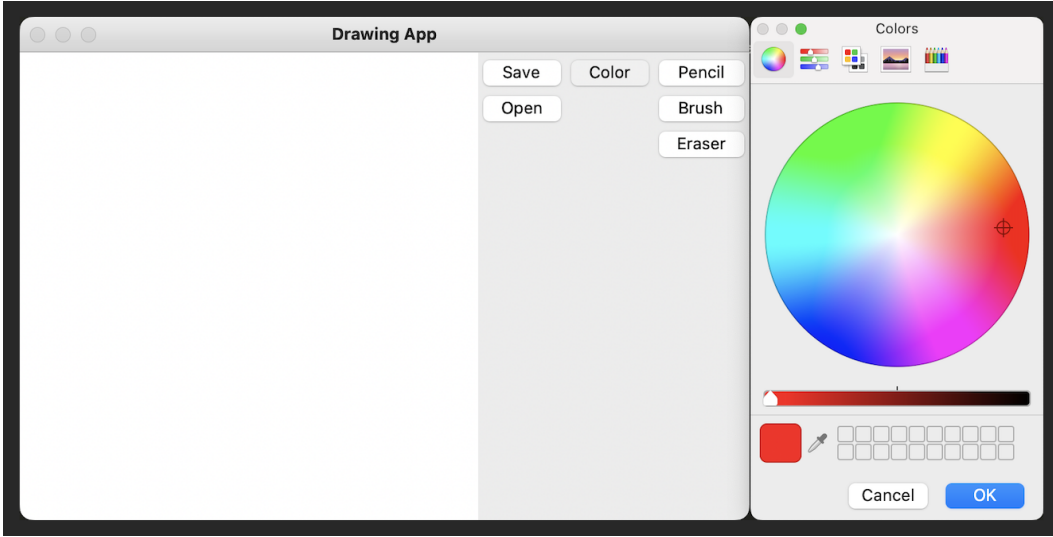


Figure 10: The “Drawing App” generated by MetaGPT.

C EXPERIMENTS

C.1 DETAILS OF THE SOFTWAREDEV DATASET

The SoftwareDev dataset includes 70 diverse software development tasks. Table 8 displays the names and detailed prompts of 11 tasks within the dataset. Note that the first seven tasks listed are used in the main experiments of this paper.

C.2 ADDITIONAL RESULTS

Quantitative results of MetaGPT As shown in Table 4, MetaGPT achieves an average score of 3.9, surpassing ChatDev’s score of 2.1 Zhao et al. (2023), which is based on the Chat chain. Compare the scores of general intelligent algorithms, including AutoGPT Torantulino et al. (2023), which all score 1.0, failing to generate executable code. We observe that the generated code is often short, lacks comprehensive logic, and tends to fail to handle cross-file dependencies correctly.

While models such as AutoGPT (Torantulino et al., 2023), Langchain (Chase, 2022), and AgentVerse (Chen et al., 2023) display robust general problem-solving capabilities, they lack an essential element for developing complex systems: systematically deconstructing requirements. Conversely, MetaGPT simplifies the process of transforming abstract requirements into detailed class and function designs through a specialized division of labor and SOPs workflow. When compared to ChatDev (Zhao et al., 2023), MetaGPT’s structured messaging and feedback mechanisms not only reduce loss of communication information but also improve the execution of code.

Quantitative results of MetaGPT w/o executable feedback Table 9 presents the performance of MetaGPT with GPT-4 32K on 11 tasks within the SoftwareDev dataset. It also shows the average performance across all 70 tasks (in the last line). Note that the version of MetaGPT used here is the basic version without the executable feedback mechanism.

Quantitative results of MetaGPT with different LLMs To verify the performance of MetaGPT on different LLMs, we randomly selected 5 SoftwareDev tasks and conducted experiments using GPT-3.5 and Deepseek Coders 33B⁵ as backends. As shown in Table 5, the results indicate that although MetaGPT can complete tasks with these LLMs, using GPT-4 as the backend yields superior performance.

⁵<https://deepseekcoder.github.io>

Table 4: **Executability comparison.** The executability scores are on a grading system ranging from '1' to '4'. A score of '1' signifies complete failure, '2' denotes executable code, '3' represents largely satisfying expected workflow, and '4' indicates a perfect match with expectations.

Task	AutoGPT	LangChain	AgentVerse	ChatDev	MetaGPT
Flappy bird	1	1	1	2	3
Tank battle game	1	1	1	2	4
2048 game	1	1	1	1	4
Snake game	1	1	1	3	4
Brick breaker game	1	1	1	1	4
Excel data process	1	1	1	4	4
CRUD manage	1	1	1	2	4
Average score	1.0	1.0	1.0	2.1	3.9

Table 5: **Performance of MetaGPT on SoftwareDev using different LLMs as agent backends.**

Model	Open source	Time(s)	# Lines	Executability	Revisions
MetaGPT (w/ GPT-3.5)	✗	75.18	161.6	2.8	2.4
MetaGPT (w/ GPT-4)	✗	552.94	178.2	3.8	1.2
MetaGPT (w/ Deepseek Coder 33B)	✓	1186.20	120.2	1.4	2.6

Impact of Instruction Levels (High-level v.s. Detailed Instructions) Does the variation in the level of initial input from humans significantly influence performance outcomes? For examples:

1. **High-level prompt:** Create a brick breaker game.
2. **Detailed prompt:** Creating a brick breaker game. In a brick breaker game, the player typically controls a paddle at the bottom of the screen to bounce a ball towards a wall of bricks. The goal is to break all the bricks by hitting them with the ball.

Additional experiments were conducted to investigate this aspect: we selected 5 tasks from SoftwareDev, and constructed detailed prompts for them. Here are the experimental results:

Table 6: **Impact of Instruction Levels.** The executability is scored on a grading system ranging from '1' to '4'. A score of '1' signifies complete failure, '2' denotes runnable code, '3' represents largely expected workflow, and '4' indicates a perfect match to expectations.

Model	# Word	Time(s)	Token usage	# Lines	Executability	Productivity	Reversions
High-level	13.2	552.9	28384.2	178.2	3.8	163.8	1.2
Detailed	42.2	567.8	29657.0	257.0	4.0	118.0	1.6

We observe that: detailed prompts lead to better software projects with lower productivity ratios because of clearer requirements and functions, while simple inputs can still generate good enough software using MetaGPT with an executability rating of 3.8, which is comparable to the detailed prompt scenario. (Note that, Productivity = Token usage / Total Code Lines. The lower this ratio, the better.)

The performance of GPT variants in HumanEval benchmark We use the GPT-4’s 67% HumanEval score (OpenAI, 2023) as our baseline, acknowledging its acceptance in the HumanEval benchmark. We further extend to experiments(five times) with GPT-4 (gpt-4-0613) and GPT-3.5-Turbo (gpt-3.5-turbo-0613) under various conditions to assess performance. (A) We directly called the OpenAI API with the prompt in HumanEval. (B) We called the OpenAI API and parsed the code with regex in the response. (C) We added an additional system prompt, then called the OpenAI API. The prompt is "You are an AI that only responds with Python code, NOT ENGLISH. You will

be given a function signature and its docstring by the user. Write your full implementation (restate the function signature).” As shown in Table 7, GPT-4 is more sensitive to prompt, code parser, and post-processing results on the HumanEval data set. It is difficult for GPT-3.5-Turbo to return the correct completion code without prompt words.

Table 7: **Performance of GPT models on HumanEval.** Experiments were conducted five times using gpt-4-0613 and gpt-3.5-turbo-0613 with different settings.

Settings	Model	1	2	3	4	5	Avg.	Std.
A	gpt-4-0613	0.732	0.707	0.732	0.713	0.738	0.724	0.013
A	gpt-3.5-turbo-0613	0.360	0.366	0.360	0.348	0.354	0.357	0.007
B	gpt-4-0613	0.787	0.811	0.817	0.829	0.817	0.812	0.016
B	gpt-3.5-turbo-0613	0.348	0.354	0.348	0.335	0.348	0.346	0.007
C	gpt-4-0613	0.805	0.805	0.817	0.793	0.780	0.800	0.014
C	gpt-3.5-turbo-0613	0.585	0.567	0.573	0.579	0.579	0.577	0.007

Qualitative results Figure 11 and Figure 12 illustrate the outcomes of the Architect agent’s efforts to design a complex recommender system. These figures showcase the comprehensive system interface design and program call flow. The latter is essential for creating a sophisticated automated system. It is crucial to emphasize the importance of this division of labor in developing an automated software framework.

D LIMITATION AND ETHICS CONCERNS

D.1 LIMITATION

System side At present, our system cannot fully cater to specific scenarios, such as UI and front-end, as we have yet to incorporate such agents and multimodal tools. Furthermore, despite generating the most amount of code among comparable frameworks, it remains challenging to fulfill real-world applications’ diverse and complex requirements.

Human user side A key challenge for users is to interrupt the running process of each agent, or set the starting running point (checkpoint) for each agent.

D.2 ETHICS CONCERNS

Unemployment and Skill Obsolescence MetaGPT enables more people to program in natural languages, thereby making it easier for engineers to get started. Over the years, programming languages have evolved from punched cards to assembly, C, Java, Python, and now natural language. As a result, humans have become more proficient at programming, increasing the demand for programming-related positions. Furthermore, programming with natural language may offer a significantly easier learning curve, making programming more accessible to a broader audience.

Transparency and Accountability MetaGPT is an open-source framework that facilitates interactive communication between multiple agents through natural language. Humans can initiate, observe, and stop running with the highest level of control. It provides real-time interpretation and operation of the natural language, displayed on the screen and logs, ensuring transparency. MetaGPT enhances “natural language programming” capabilities, and human engineers are the users and responsible for the outcomes.

Privacy and Data Security MetaGPT operates locally, ensuring user data privacy and security. It does not collect user data. For interactions with third-party LLMs, such as those by OpenAI, users are encouraged to refer to the respective privacy policies (e.g., OpenAI Privacy Policy). However, we provide the option of open-source LLMs as backends.